

Applicazione alla complessità aritmetica

Definizione 1. Un **algoritmo di moltiplicazione veloce** è un algoritmo che usa asintoticamente un numero di moltiplicazioni rispetto alle addizioni.

Definizione 2 (Moltiplicazione matriciale). Fissati degli interi positivi m, n, p , la moltiplicazione tra due matrici $m \times n$ e $n \times p$ è una mappa bilineare

$$\langle m, n, p \rangle : \mathbb{C}^{m \times n} \times \mathbb{C}^{n \times p} \rightarrow \mathbb{C}^{m \times p}$$
$$(A, B) \mapsto C = AB.$$

Spesso ci riferiremo a $\langle m, n, p \rangle$ chiamandolo «algoritmo», ad esempio nel contesto degli algoritmi di moltiplicazione veloce.

Domanda:

Possiamo descrivere un algoritmo di moltiplicazione con un tensore e sperare di ridurre la complessità aritmetica?

(risposta sì)

Bilinear computation

Data una mappa bilineare $\varphi : U \times V \rightarrow W$, per ogni $r = 1, \dots, R$ siano $f_r \in U^*$, $g_r \in V^*$, $w_r \in W$ tali che

$$\varphi(u, v) = \sum_{r=1}^R f_r(u)g_r(v)w_r,$$

per ogni $u \in U, v \in V$.

La r -upla $(f_1, g_1, w_1, \dots, f_r, g_r, w_r)$ è detta **bilinear computation**.
La lunghezza di una bilinear computation è detta **bilinear complexity (o rango)** di φ , e si indica con $R(\varphi)$.

Rappresentazione low-rank di forme bilineari

Proposizione 1. *Data una decomposizione*

$$T = \sum_{r=1}^R a_r \otimes b_r \otimes c_r$$

Possiamo scrivere una forma bilineare $\varphi : U \times V \rightarrow W$ come una bilinear computation della forma

$$\varphi(u, v) = \sum_{r=1}^R (a_r^\top u) (b_r^\top v) c_r.$$

Rappresentazione low-rank del prodotto matriciale

Date due matrici $A \in \mathbb{R}^{m \times n}$ e $B \in \mathbb{R}^{n \times p}$, poste $u = \text{vec}(A)$, $v = \text{vec}(B)$ abbiamo un algoritmo di moltiplicazione veloce a partire da una bilinear computation

$$\text{vec}(C^\top) = \varphi(u, v) = \sum_{r=1}^R \underbrace{(a_r^\top u)}_{f_r} \cdot \underbrace{(b_r^\top v)}_{g_r} c_r,$$

Active multiplications

Il rango

So far:

$$\begin{aligned} R(\varphi) &= \text{"Lunghezza di una bilinear computation"} \\ &= \text{"Rango del tensore associato"} \\ &= \text{"Numero di moltiplicazioni matriciali"} \end{aligned}$$

Vorremmo mostrare che

$$R(\varphi) = .$$

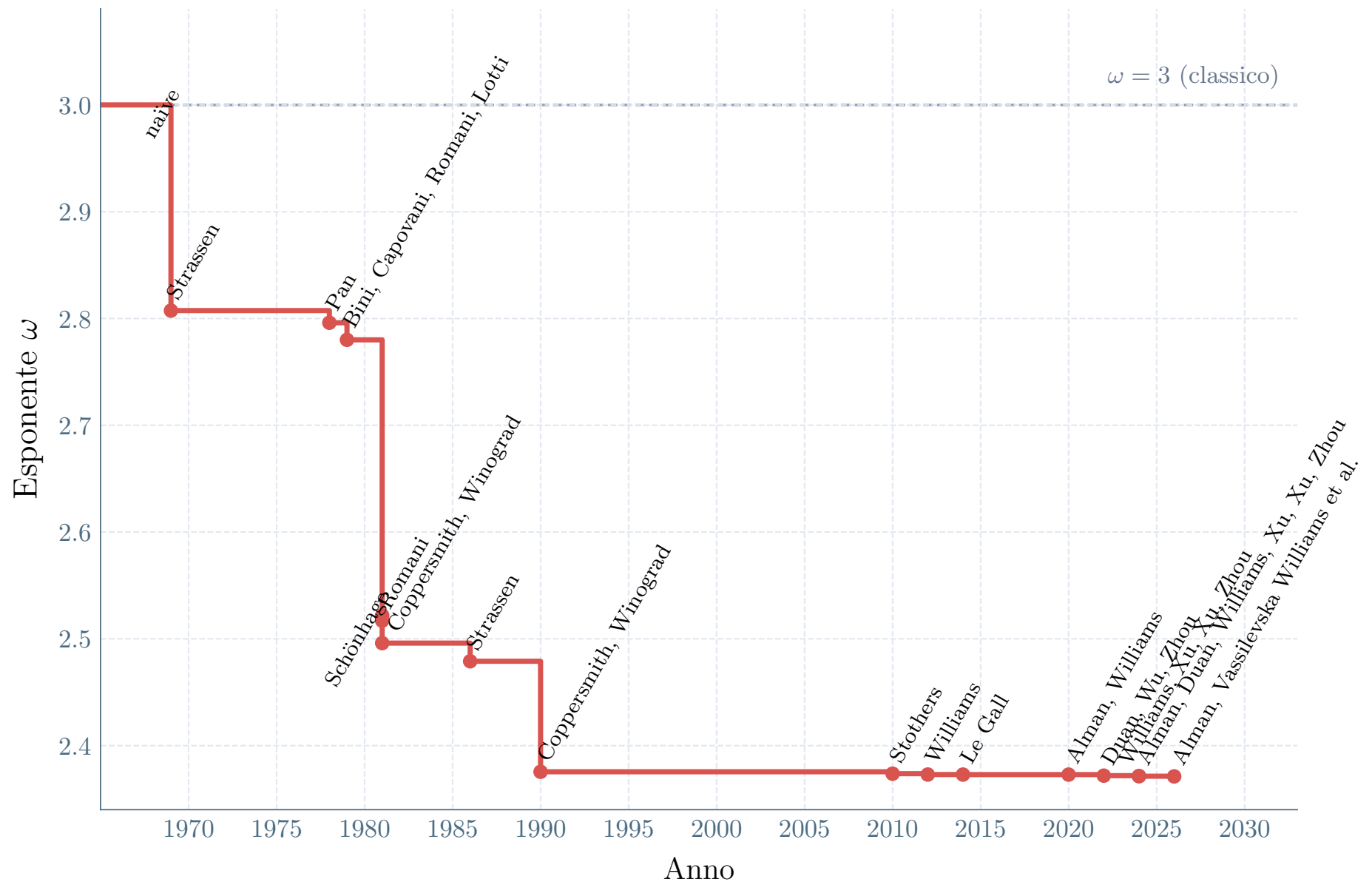
Spoiler: equazioni $\rightarrow$ biliner comp $\rightarrow$ tensore

L'esponente

Definizione 1. L'esponente della moltiplicazione matriciale è il numero

$$\omega := \inf_n \{ \tau \in \mathbb{R} \mid \text{La moltiplicazione tra due matrici in } \mathbb{R}^{n \times n} \\ \text{ha costo } o(n^\tau) \}.$$

Stime note su ω



L'algoritmo classico

$$\begin{aligned} \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} &= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix} \\ &= \begin{bmatrix} f_1g_1 + f_2g_2 & f_3g_3 + f_4g_4 \\ f_5g_5 + f_6g_6 & f_7g_7 + f_8g_8 \end{bmatrix} \\ &= \begin{bmatrix} m_1 + m_2 & m_3 + m_4 \\ m_5 + m_6 & m_7 + m_8 \end{bmatrix} \\ &= \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} \end{aligned}$$

Fissata la base standard $E_{11}, E_{12}, E_{21}, E_{22}$ dello spazio delle matrici 2×2 , possiamo scrivere la bilinear computation

$$\begin{aligned} C &= c_{11}E_{11} + c_{12}E_{12} + c_{21}E_{21} + c_{22}E_{22} \\ &= (m_1 + m_2)E_{11} + (m_3 + m_4)E_{12} \\ &\quad + (m_5 + m_6)E_{21} + (m_7 + m_8)E_{22} \\ &= \sum_{r=1}^8 m_r w_r. \end{aligned}$$

L'algoritmo corrisponde al tensore

$$\begin{aligned} \langle 2, 2, 2 \rangle &= (a_{11} \otimes b_{11} + a_{12} \otimes b_{21}) \otimes c_{11} \\ &\quad + (a_{21} \otimes b_{11} + a_{22} \otimes b_{21}) \otimes c_{21} \\ &\quad + (a_{11} \otimes b_{12} + a_{12} \otimes b_{22}) \otimes c_{12} \\ &\quad + (a_{21} \otimes b_{12} + a_{22} \otimes b_{22}) \otimes c_{22}. \end{aligned}$$

L'algoritmo di Strassen

Facendo alcune semplificazioni del risultato del prodotto matriciale, troviamo l'algoritmo di Strassen in forma tensoriale

$$\begin{aligned}\langle 2, 2, 2 \rangle = & (a_{11} + a_{22}) \otimes (b_{11} + b_{22}) \otimes (c_{11} + c_{22}) \\ & + (a_{21} + a_{22}) \otimes b_{11} \otimes (c_{21} - c_{22}) \\ & + a_{11} \otimes (b_{12} - b_{22}) \otimes (c_{12} + c_{22}) \\ & + a_{22} \otimes (-b_{11} + b_{21}) \otimes (c_{21} + c_{11}) \\ & + (a_{11} + a_{12}) \otimes b_{22} \otimes (-c_{11} + c_{12}) \\ & + (-a_{11} + a_{21}) \otimes (b_{11} + b_{12}) \otimes c_{22} \\ & + (a_{12} - a_{22}) \otimes (b_{21} + b_{22}) \otimes c_{11}.\end{aligned}$$

Che ha rango 7.

L'algoritmo di Strassen

Due domande:

Come calcolare la complessità quando si usano i due algoritmi ricorsivamente?

Come ridurre il rango migliora la complessità?

Stimare la complessità nel caso generale

Date due matrici di taglia $2^i \times 2^i$ possiamo fare i livelli di ricorsione e ottenere che

$$R(\langle 2^i, 2^i, 2^i \rangle) = R(\langle 2, 2, 2 \rangle^{\otimes i}) \leq R(\langle 2, 2, 2 \rangle)^i \leq r^i.$$

Nel caso generale, dato $n > 2$, possiamo fare un padding di zeri
TODO: sistemare per farlo sembrare un embedding dentro matrice più grande

$$\begin{bmatrix} A & \end{bmatrix} \begin{bmatrix} B & \end{bmatrix}$$

fino a raggiungere la taglia $2^{\lceil \log_2 n \rceil}$.

La complessità sarà comunque limitata da R :

$$\begin{aligned} \mathbf{R}(\langle n, n, n \rangle) &\leq \mathbf{R}(\langle 2^{\lceil \log_2 n \rceil}, 2^{\lceil \log_2 n \rceil}, 2^{\lceil \log_2 n \rceil} \rangle) \\ &\leq \mathbf{R}(\langle 2, 2, 2 \rangle)^{\lceil \log_2 n \rceil} \\ &\leq R^{\lceil \log_2 n \rceil} \\ &\leq R \cdot n^{\log_2 R} \end{aligned}$$

Da cui

Proposizione 1. *Se $\mathbf{R}(\langle n, n, n \rangle) \leq R$ per degli interi positivi n, R , allora $n^\omega \leq R$.*

Abbiamo dimostrato che $\omega \leq \log_n \mathbf{R}(\langle n, n, n \rangle)$.

Da cui,

$$\omega \leq \inf_n \log_n \mathbf{R}(\langle n, n, n \rangle).$$

Applicazione all'algoritmo di Strassen

Se $R = 8$, dalla Proposizione 1 troviamo

$$\omega \leq \log_2 \mathbf{R}(\langle 2, 2, 2 \rangle) = \log_2 8 = 3.$$

L'algoritmo di Strassen migliora questa stima in quanto

$$\omega \leq \log_2 \mathbf{R}(\langle 2, 2, 2 \rangle) = \log_2 7 \approx 2.81.$$

Che è il bound più basso per ω ottenibile da $\langle 2, 2, 2 \rangle$.

Ad esempio, Bini et Al. ^[1] trovano che $\mathbf{R}(12) = 100$, da cui

$$\omega \leq \log_{12} 1000 \approx 2.7799.$$

^[1]D. Bini, M. Capovani, F. Romani, e G. Lotti, «O (n^{2.7799}) complexity for n\times n approximate matrix multiplication», *Information processing letters*, vol. 8, fasc. 5, pp. 234–235, 1979.

Il teorema di Strassen

Vorremmo che

$$\omega = \liminf_{n \rightarrow +\infty} \log_n \mathbf{R}(\langle n, n, n \rangle).$$

Teorema 1 (Strassen ^[2] ^[3]).

$$\omega = \inf_n \{ \tau \in \mathbb{R} \mid \mathbf{R}(\langle n, n, n \rangle) = O(n^\tau) \}.$$

Ovvero, è possibile moltiplicare due matrici $n \times n$ usando $O(n^{\omega+\varepsilon})$ operazioni aritmetiche se e solo se $\mathbf{R}(\langle n, n, n \rangle) = O(n^{\omega+\varepsilon})$.

^[2]V. Strassen, «Gaussian elimination is not optimal», *Numer. Math.*, vol. 13, fasc. 4, pp. 354–356, ago. 1969, doi: 10.1007/BF02165411.

^[3]P. Bürgisser, M. Clausen, e M. A. Shokrollahi, *Algebraic complexity theory*, vol. 315. Springer Science & Business Media, 2013.

Algoritmi approssimati (se c'è tempo)

bini et al provano...

ensore che trovano

Cenni sull'errore

Bibliografia

- [1] D. Bini, M. Capovani, F. Romani, e G. Lotti, « $O(n^2 \cdot 7799)$ complexity for $n \times n$ approximate matrix multiplication», *Information processing letters*, vol. 8, fasc. 5, pp. 234–235, 1979.
- [2] V. Strassen, «Gaussian elimination is not optimal», *Numer. Math.*, vol. 13, fasc. 4, pp. 354–356, ago. 1969, doi: 10.1007/BF02165411.
- [3] P. Bürgisser, M. Clausen, e M. A. Shokrollahi, *Algebraic complexity theory*, vol. 315. Springer Science & Business Media, 2013.